Project: Predicting Dormancy in a Retail Brokerage

I wrote and ran the data generator and a prototype of the cleaning, labeling and modeling below. The "expected results" in each stage come from that run, so you can check your own work against them. Your numbers should match closely if you make the same cleaning choices.

Part 1: The dataset

What it is

A synthetic retail brokerage with 5,000 accounts and 24 monthly snapshots (Jan 2023 to Dec 2024), split across two raw files:

- **accounts.csv** has one row per account: open date, age, account type, acquisition channel, risk tolerance, stated income.
- **monthly_activity.csv** has one row per account-month: end-of-month balance, logins, trades, deposits, withdrawals, fees, monthly return.

The generator builds in a hidden engagement trait, a bear market (Sep 2023 to Jan 2024), and a churn process in which at-risk accounts fade for one month and then go silent. Those are the patterns your model has to find. It also injects the messiness listed in the table in Stage 2.

Where this data comes from in the real world

Brokerages would pull this from a CRM or account-master table, monthly statement data, and app event logs, joined in a warehouse. No public brokerage-behavior dataset exists, so for a public version:

- **Market returns:** replace the simulated market factor with real monthly S&P 500 returns from Yahoo Finance (yfinance, ticker ^GSPC). Join them to the activity table to build a real market-drawdown feature.
- **Churn framing:** the Kaggle "Credit Card Customers" (BankChurners) dataset is a public churn dataset with the same structure.

Generator script (src/generate_data.py)

Run from the repo root: python src/generate_data.py. Requires numpy and pandas.

python

```
"""Generate a synthetic retail-brokerage dataset (fixed seed => reproducible).

Outputs (data/raw/):
  accounts.csv         one row per account (+ a few duplicate rows)
  monthly_activity.csv   one row per account-month, Jan-2023..Dec-2024
"""
from pathlib import Path
import numpy as np
import pandas as pd

SEED = 42
N = 5000
N_MONTHS = 24
rng = np.random.default_rng(SEED)
OUT = Path("data/raw"); OUT.mkdir(parents=True, exist_ok=True)
months = pd.period_range("2023-01", periods=N_MONTHS, freq="M")

# ---------------------------------------------------------------  accounts
acct_id = np.array([f"ACC{i:05d}" for i in range(1, N + 1)])
open_date = pd.Timestamp("2023-01-01") + pd.to_timedelta(
    rng.integers(-36, N_MONTHS - 2, N) * 30.4 + rng.integers(0, 28, N), unit="D")
open_date = pd.Series(open_date).dt.floor("D")
open_month = ((open_date.dt.year - 2023) * 12 + open_date.dt.month - 1).to_numpy()

age = np.clip(rng.normal(41, 12, N), 18, 85).round()
account_type = rng.choice(["Cash", "Margin", "IRA"], N, p=[.55, .25, .20])
```

```
channel = rng.choice(["Mobile App", "Web", "Referral", "Advisor Lead"], N, p=[.45, .30, .15, .10])
risk_tol = rng.integers(1, 6, N)
income = np.exp(rng.normal(11.1, 0.5, N)).round(-3)
engagement = rng.gamma(2.0, 1.0, N)
engagement = np.clip(engagement / engagement.mean(), 0.05, None)  # latent trait, never exported
beta = np.clip(rng.normal(1.0 + 0.1 * (risk_tol - 3), 0.25), 0.2, 2.2)
init_bal = np.exp(rng.normal(8.8, 1.1, N))

# ------------------------------------------------------------  simulation
mkt = rng.normal(0.008, 0.035, N_MONTHS)
mkt[8:13] = rng.normal(-0.035, 0.03, 5)       # bear stretch: Sep-2023 .. Jan-2024

cur = np.zeros(N); started = np.zeros(N, bool)
fade_start = np.full(N, -1)
ret_hist = np.zeros((N_MONTHS, N))
rows = []
for m in range(N_MONTHS):
    live = open_month <= m
    new = live & ~started
    cur[new] = init_bal[new]; started |= new
    k = np.where(fade_start >= 0, m - fade_start, -1)
    mult = np.where(k < 0, 1.0, np.where(k == 0, 0.35, 0.0))  # churners fade, then go silent

    logins = rng.poisson(6.0 * engagement * mult) * live
    trades = rng.poisson(2.2 * engagement * (0.8 + 0.1 * risk_tol) * mult) * live
```

```
    ret = beta * mkt[m] + rng.normal(0, 0.02, N)
    ret_hist[m] = ret * live
    dep = (rng.random(N) < np.clip(0.2 * engagement * mult, 0, .9)) * rng.lognormal(6.0, 0.8, N) * live
    wd_p = np.where((k >= 0) & (k <= 1), 0.5, 0.04)          # leavers pull money out
    wd = (rng.random(N) < wd_p) * rng.uniform(0.1, 0.9, N) * cur * live
    fees = trades * 0.65
    cur = np.where(live, np.maximum(cur * (1 + ret) + dep - wd - fees, 0), 0)

    dd = ret_hist[max(0, m - 2): m + 1].sum(axis=0)
    tenure = m - open_month
    fee_ratio = fees / np.maximum(cur, 50)
    logit = (-4.4 - 0.8 * np.log(engagement) + 6.0 * np.clip(-dd, 0, 0.3)
             + 0.7 * (tenure < 6) + 0.6 * (fee_ratio > 0.002)
             + 0.25 * (channel == "Mobile App") - 0.4 * (channel == "Advisor Lead"))
    trig = live & (fade_start < 0) & (rng.random(N) < 1 / (1 + np.exp(-logit)))
    fade_start[trig] = m + 1

    rows.append(pd.DataFrame({
        "account_id": acct_id[live], "m": m, "balance_usd": cur[live].round(2),
        "logins": logins[live], "trades": trades[live],
        "deposits_usd": dep[live].round(2), "withdrawals_usd": wd[live].round(2),
        "fees_usd": fees[live].round(2), "monthly_return_pct": (100 * ret[live]).round(2)}))

act = pd.concat(rows, ignore_index=True)
act["month"] = np.array(months.strftime("%Y-%m"))[act.pop("m").to_numpy()]
```

```
# ---------------------------------------------  inject messiness: activity

act = act.drop(act.sample(frac=0.01, random_state=SEED).index).reset_index(drop=True)  # missing months

n = len(act)

act.loc[rng.random(n) < .02, "balance_usd"] = np.nan                    # missing balances

act.loc[rng.choice(n, 15, replace=False), "balance_usd"] *= 100         # decimal-shift typos

atype = dict(zip(acct_id, account_type))

cand = act.index[act["account_id"].map(atype).isin(["Cash", "IRA"])]

neg = rng.choice(cand, 20, replace=False)

act.loc[neg, "balance_usd"] = -act.loc[neg, "balance_usd"].abs()        # impossible negatives

act.loc[rng.random(n) < .005, "logins"] = -1                            # sentinel for "unknown"

act["trades"] = act["trades"].astype(object)

act.loc[rng.choice(n, 25, replace=False), "trades"] = "n/a"             # text in numeric col

alt = rng.random(n) < .04

act.loc[alt, "month"] = pd.to_datetime(act.loc[alt, "month"]).dt.strftime("%b-%Y")      # mixed date formats

act = pd.concat([act, act.sample(25, random_state=SEED)], ignore_index=True)          # exact duplicates


# ---------------------------------------------  inject messiness: accounts

fmts = ["%Y-%m-%d", "%m/%d/%Y", "%d %b %Y"]

pick = rng.choice(3, N, p=[.6, .25, .15])

open_str = [d.strftime(fmts[c]) for d, c in zip(open_date, pick)]
```

```
variants = {"Mobile App": ["mobile app", "MOBILE APP", "App", "mobile_app"],
        "Web": ["web", "WEB "], "Referral": ["referral", "Ref."],
        "Advisor Lead": ["advisor lead", "Advisor-Lead"]}
channel_raw = [rng.choice(variants[c]) if rng.random() < .35 else c for c in channel]

fmt_mask = rng.random(N) < .30
inc = pd.Series([f"${v:,.0f}" if f else v for v, f in zip(income, fmt_mask)], dtype=object)
inc[rng.random(N) < .12] = np.nan

age[rng.random(N) < .03] = np.nan
age[rng.choice(N, 12, replace=False)] = rng.choice([0, 2, 118, 150, 999], 12)        # impossible ages
risk = risk_tol.astype(float); risk[rng.random(N) < .08] = np.nan

accts = pd.DataFrame({"account_id": acct_id, "open_date": open_str, "age": age,
                "account_type": account_type, "channel": channel_raw,
                "risk_tolerance": risk, "stated_income": inc})
accts = pd.concat([accts, accts.sample(40, random_state=SEED)], ignore_index=True)     # duplicate accounts

accts.to_csv(OUT / "accounts.csv", index=False)
act.to_csv(OUT / "monthly_activity.csv", index=False)
print(f"accounts: {accts.shape}, activity: {act.shape}")
```

Expected output: accounts: (5040, 7), activity: (98982, 9).

Part 2: Project framing

Question. *Which active brokerage accounts are likely to go dormant within the next 90 days, and how much of the at-risk assets could a retention team protect if it can only contact the riskiest 10%?*

Audience. A head of client retention or an analytics manager at a brokerage. They care about how many accounts the model catches and what a limited call budget buys, not about AUC.

The story. Dormancy is rare (about 10% of active accounts per quarter) and costly, because dormant accounts often precede withdrawals. I built a 90-day early-warning model on monthly behavioral data, evaluated it on a later time period rather than a random split, and compared it with simple baselines. Declining engagement and recent portfolio losses are the main warning signs. The model finds roughly 4× more at-risk accounts than random outreach in the top decile. Market conditions changed between training and test, so the drawdown signal is the part I'd trust least without more data.

Why it works as a portfolio piece. It is a rare-event problem with a time dimension and an operational constraint, and the rigor lives in the choices: label definition, leakage prevention, temporal validation, and the metric. Those are the things hiring managers look for. It also draws on your bioinformatics habits: class imbalance, leakage, and honest evaluation.

Stage 1: Data loading and inspection

Do:

1. Load both CSVs with pd.read_csv, with no dtype coercion yet.
2. For each frame, print .shape, .dtypes, .head(), .isna().sum(), .nunique(), and .describe(include="all").
3. Write down what the inspection should reveal before you clean anything:
    - trades and stated_income load as object when they should be numeric.
    - account_id has more rows than unique values in both files.
    - channel has 12 or more distinct strings for 4 real categories.
    - age has a minimum of 0 and a maximum of 999.
    - balance_usd has a negative minimum and a maximum around 100× the 99th percentile.

Deliverable: notebooks/01_inspection.ipynb ending in a markdown cell, “Issues found,” with a numbered list. That list is your Stage 2 checklist and shows the reviewer you diagnosed before treating.

Stage 2: Cleaning and validation

Do each fix in a function in src/cleaning.py, and log row counts before and after each step.

Issue	Handling	Why (tradeoff)
40 duplicate account rows, ~25 duplicate activity rows	drop_duplicates(), then assert account_id is unique and (account_id, month) is unique	These are exact copies, so dropping is safe. Assert afterward so a silent re-introduction fails loudly.
open_date in 3 formats	pd.to_datetime(…, format="mixed")	Slash dates like 03/04/2023 are ambiguous (US vs. day-first). Assume US and say so in the README.
month as 2023-01 and Jan-2023	Parse to Period[M]	A period type gives clean month arithmetic and joins.
channel spelled 12+ ways	.str.strip().str.lower(), then an explicit mapping dict. Assert exactly 4 values remain.	Use an explicit map, not fuzzy matching. App → mobile app and Ref. → referral are judgment calls you can document.
stated_income as "$85,000" and floats	Strip $ and ,, then pd.to_numeric. About 12% stays missing.	Don’t impute for display or EDA. For modeling, use median imputation plus a income_missing flag, since missingness may carry signal.
age of 0, 2, 118, 150, 999	Set values outside 18–100 to NaN	Brokerage accounts require adults, so these are typos. Setting to NaN is more honest than guessing.

Issue	Handling	Why (tradeoff)
risk_tolerance ~8% missing	Keep as NaN, then impute in the modeling pipeline only	Imputing inside the pipeline keeps test-set statistics out of training.
logins = -1	Set to NaN	The value is a sentinel for "unknown," not a count. Averaging it would corrupt your activity features.
trades contains "n/a"	pd.to_numeric(errors="coerce")	Treat as unknown, not zero. Zero would falsely make an account look inactive and contaminate your label.
Negative balances (~20 rows) on Cash/IRA accounts	Set to NaN	These account types can't go negative. Margin accounts legitimately can, which is why the rule is conditional on account type, not a blanket >= 0 filter.
Balance typos (~15 rows at 100× normal)	Flag a balance greater than 20× the centered 5-month rolling median for that account, then set to NaN	Do not clip at a percentile. Real balances are heavy-tailed, so a large account is not an error. A value that is huge relative to *that account's own neighbors* and then reverts is. My rule caught 14 of 15 injected typos. Investigate the miss and write it up.
Missing account-months (~1%)	Reindex to the full account × month grid, so gaps are explicit NaN rows	A missing month is not a zero-activity month. This distinction protects your label in Stage 5.

Validation block (end of cleaning, as assert statements):

- no duplicate keys
- age is between 18 and 100 or null
- channel has 4 unique values

- no negative balances for Cash/IRA
- month range is 2023-01 to 2024-12

Write the cleaned data to data/processed/ as Parquet or CSV.

Stage 3: Exploratory analysis in pandas

Each item below is a specific computation, not general exploration.

1. **Activity definition.** Add activity = trades.fillna(0) + logins.fillna(0). Define a *zero-activity month* as activity == 0 where both inputs are known.
2. **Monthly health table** using groupby("month"): active accounts, total AUM, mean logins, mean trades, and the share of accounts with zero activity. Use .agg() with named aggregation.
3. **Rolling and time operations.** Compute a 3-month rolling sum of logins and trades per account with groupby().transform(rolling). Compute month-over-month balance change with pct_change(). Use resample("Q") to get quarterly AUM and net flows (deposits minus withdrawals).
4. **Tenure effect.** Bucket tenure into 0–5, 6–11, 12–23, and 24+ months with pd.cut. Then compute the rate of first zero-activity month by bucket.
5. **Segment cuts** with groupby(["account_type", "channel"]): dormancy rate, median balance, and median logins. Add a pivot_table.
6. **Cohort retention.** Group accounts by open quarter and build a pivot (rows are cohorts, columns are months since open) of the share still active. You will use this for a heatmap.
7. **Drawdown test.** Compute each account's trailing 3-month cumulative return. Bucket it (≤ −10%, −10 to −3, −3 to 3, > 3) and compare the later dormancy rate across buckets.
8. **Dormancy concentration.** Check what share of total AUM sits in accounts that later go dormant, to size the dollars at stake.

Write a one-line takeaway under each result as you go. They become the raw material for Stage 6.

Stage 4: Visualization in matplotlib

Use fig, ax = plt.subplots(…), not the plt. shortcuts. Give every chart a headline title that states the finding, labeled axes with units, and a source note. Save each at dpi=200 to reports/figures/. Make exactly these six:

#	Plot	Shows	Labeling and design
1	Line chart: share of active accounts with zero activity, by month	Dormancy trend over time	Shade Sep 2023 to Jan 2024 with axvspan and label it “Simulated bear market”. Title example: “Dormancy rose during the market downturn.”
2	Bar chart: dormancy rate by tenure bucket	New accounts are more fragile	Annotate each bar with its percentage. Order buckets chronologically. Use a single color.
3	Bar chart: dormancy rate by trailing 3-month return bucket, with n under each label	Drawdown relationship	Put the sample size in each tick label, because the sparse buckets are the honest caveat.
4	Heatmap (imshow): cohort retention	How retention decays by cohort	Colorbar labeled “% still active”. Mask the upper-right triangle where data doesn’t exist.
5	Two-panel figure: precision-recall curve (left) and cumulative capture curve (right)	Model quality in business terms	Put the baseline and the random line on both. On the capture chart, mark the top-10% point with a callout.
6	Horizontal bar chart: permutation importance on the test set, with error bars	What drives predictions	Sort descending. Use plain English labels (e.g. “Change in activity vs. prior 3 months”), not column names.

Plots 1–4 belong in Stage 4. Plots 5 and 6 need the model, so you build them in Stage 5 and save them to the same folder.

Stage 5: Machine learning

The modeling problem

Binary classification at the account-month level. At each snapshot month *t*, the features come from months ≤ *t*, and the label looks forward over *t*+1 to *t*+3.

Label: y = 1 if the account has at least one zero-activity month in *t*+1 to *t*+3.

Population rules:

- Include only accounts with activity > 0 in month *t*. An account already dormant is not a prediction problem, and including it would inflate every metric.
- Include only snapshots where all three future months are known (not missing). This is why Stage 2 kept missing months as NaN.

python

```
g = panel.groupby("account_id")

future = pd.concat([g["activity"].shift(-i) for i in (1, 2, 3)], axis=1)

panel["y"] = (future.min(axis=1) == 0).astype(int)
```

Expected result: about 65,000 labelable account-months with a base rate near 11%.

Features (all computed from data ≤ *t*)

Trailing 3-month logins, trades, deposits, withdrawals, fees, and return. Current-month activity. An activity trend (current month ÷ (prior 3-month average + 1)). Tenure in months. Balance. Withdrawal ratio (withdrawals ÷ balance). Fee ratio (fees ÷ balance). Account type, channel, risk tolerance, age, and income with a missing flag.

Leakage checklist, to put in the README: no feature uses any month after *t*; imputation and scaling are fit inside a sklearn Pipeline on training data only; the latent engagement trait is not in the data.

Split: temporal, with a gap

- **Train:** snapshots Jun 2023 to Mar 2024 (about 31,000 rows).
- **Gap:** Apr to Jun 2024. The last training labels reach into June, so test snapshots start after that.
- **Test:** snapshots Jul to Sep 2024 (about 9,500 rows). Labels run through Dec 2024.

Tradeoff: a random split is simpler and gives higher scores, but it lets the model learn from the future, which doesn't match how it would be used. Use a time split and say why.

Tune with expanding-window time-series CV *inside* the training period. Never tune on the test set.

Models

1. **Naive baseline:** rank accounts by the activity trend alone. Expect AUC ≈ 0.61.
2. **Logistic regression** (median imputation and standard scaling, in a Pipeline). It is interpretable, and the coefficients are explainable to a non-technical audience.
3. **HistGradientBoostingClassifier** (max_depth=3, learning_rate=0.05, ~300 iterations). It handles missing values and nonlinearities natively.

Why two real models: in my run, logistic regression reached AUC ≈ 0.79 and gradient boosting reached ≈ 0.80, which is nearly identical. Report that honestly. The gap is too small to justify the complexity, so recommending the simpler model is the stronger judgment call.

Metrics

- **Primary: PR-AUC.** With a ~10% base rate, ROC-AUC looks flattering. Expected: ≈ 0.32 (logistic) and ≈ 0.37 (boosting), against a baseline around 0.18.
- **Business metric: precision and capture in the top 10% of risk scores**, because the retention team can only call so many accounts. Expected: top-decile precision ≈ 40%, capturing ≈ 42% of dormancy cases, which is ≈ 4.2× lift.
- **Also report:** ROC-AUC (≈ 0.79–0.80) and a calibration plot. Don't tune the threshold to "maximize F1," since the real threshold is set by call capacity.
- **Uncertainty:** bootstrap the test set by *account*, not by row, with 1,000 resamples. Each account appears in up to three snapshots, so row-level resampling understates the uncertainty. Report 95% intervals.

Interpretation

- Use **permutation importance** on the test set (plot 6). It is more reliable than built-in importances.
- **Regime-shift check, which is where the analytical judgment shows.** The training period includes a bear market, but the test period is calm. In my run, only 125 test snapshots fall in the ≤ −3% return bucket, so the drawdown effect is poorly measured out-of-sample. State this plainly. Don't claim the model "learned the drawdown effect" without qualification.

- **Honest framing:** the data is synthetic, so the project shows the *method and judgment*, not real market behavior. Say so in the README.

Stage 6: Findings and write-up

Fill in the final wording only after your own run. Use this skeleton.

Key conclusions (a hiring manager should get these in under two minutes):

1. About 1 in 10 active accounts goes dormant within a quarter, and newer accounts are more fragile.
2. Falling activity relative to an account's own recent baseline is the strongest early signal, though a naive trend rule alone is weak (AUC ≈ 0.61).
3. A logistic regression performs about as well as gradient boosting (AUC ≈ 0.79 vs. 0.80). I recommend the simpler model because it is easier to explain and monitor.
4. Contacting only the riskiest 10% of accounts reaches about 4× the dormancy rate of random outreach, and about two-fifths of all dormancy cases.
5. The drawdown effect appears in training data but is weakly testable in the calm test period.

Limitations (be direct):

- The data is synthetic, so the relationships are modeled, not discovered.
- Dormancy is a proxy. It isn't the same as account closure or lost revenue.
- The label ignores whether a retention call would actually *change* the outcome. That would require an uplift model or an experiment.
- The test window is only three snapshot months and one market regime.
- Observations from the same account are correlated. I addressed this with an account-level bootstrap, but it is still a limit on how much to read into the numbers.

Next steps:

- Replace the simulated market factor with real S&P 500 returns from yfinance.
- Add a cost-weighted ranking: expected dollars at risk (probability × balance) instead of probability alone.

- Design an A/B test for the retention outreach, and add survival analysis (time-to-dormancy).

Part 4: Repository structure

```
brokerage-dormancy-model/
├── README.md
├── requirements.txt
├── .gitignore              # ignore data/ except a small sample
├── data/
│   ├── raw/                # generated; not committed (script reproduces it)
│   ├── processed/
│   └── sample/             # 100-row samples of each raw file (committed)
├── notebooks/
│   ├── 01_inspection.ipynb
│   ├── 02_cleaning.ipynb
│   ├── 03_eda.ipynb
│   └── 04_modeling.ipynb
├── src/
│   ├── generate_data.py
│   ├── cleaning.py
│   ├── features.py         # snapshot panel, labels, trailing features
│   └── modeling.py         # train/eval functions, bootstrap CI
├── reports/
│   ├── figures/            # the six PNGs
│   └── summary.md          # the one-page findings
└── tests/
```

└── test_cleaning.py # 4–5 asserts: unique keys, valid ranges, label has no future leakage

README contents, top to bottom:

1. One-sentence question and a 3–4 bullet "Key findings" block with the numbers.
2. A "Synthetic data" note, with where real data would come from and the public substitutes.
3. Your best figure (plot 5 or 3) embedded.
4. Method summary: label definition, temporal split with gap, metrics, and why.
5. Limitations and next steps.
6. Reproduce: pip install -r requirements.txt, python src/generate_data.py, and the order to run the notebooks.

Move the logic out of the notebooks and into src/, so the notebooks mostly call functions and narrate results. Reviewers who open src/ should see readable code, and the tests show you validate data.

Part 5: Resume bullets

Replace the bracketed numbers with your own results. Keep "synthetic" or "simulated" in at least one bullet, since hiring managers value honesty.

- Built an end-to-end early-warning model for brokerage account dormancy on a simulated 5,000-account, 24-month panel, framing a 90-day forward label that excludes already-dormant accounts to avoid inflated metrics.
- Cleaned ~99K activity records in pandas, resolving [11] data-quality issues (mixed date formats, sentinel values, unit-shift outliers, duplicates) with explicit validation checks and unit tests.
- Evaluated logistic regression and gradient boosting using a time-based split with a 3-month purge gap, reporting PR-AUC [0.32–0.37] and account-level bootstrap intervals; showed the simpler model matched boosting (AUC [0.79 vs. 0.80]).
- Translated model output into a capacity-constrained outreach policy: contacting the top 10% of accounts captured [~42%] of dormancy cases at [~4×] lift over random targeting, and flagged regime-shift risk in the drawdown signal.

Part 6: Seven-day plan

Day	Focus	Done when
1	Create the repo, run the generator, complete Stage 1 inspection and the "Issues found" list	Raw files load and the issues list is written
2	Stage 2 cleaning in src/cleaning.py, with validation asserts and tests	Clean data saved, all asserts pass
3	Stage 3 EDA: groupbys, rolling/resample operations, cohort pivot, takeaways	Eight computations done, each with a one-line takeaway
4	Stage 4 plots 1–4, plus build the snapshot panel, label, and features (features.py)	Four figures saved; label base rate checked (~11%)
5	Stage 5 modeling: baseline, logistic regression, boosting, time-series CV, test evaluation	Metrics table complete; no test-set tuning
6	Bootstrap CIs, permutation importance, calibration, regime-shift check, plots 5–6	Every claim in Stage 6 backed by a number
7	README, summary.md, tidy notebooks, run everything fresh from a clean clone, record resume bullets	A stranger can reproduce your results from the README

Days 5 and 6 are both for modeling, so you won't rush the part reviewers weigh most. If you fall behind, cut plot 4 (the heatmap) and the calibration plot, not the temporal split or the bootstrap.